ECSE 458 Capstone Design Project

DP 06 – Project Documentation

Written By

Hien Le (261118519)

Project Advisor

Professor David A. Lowther (david.lowther@mcgill.ca)

Company

The Computational Electromagnetics Research Group at McGill

Table of Contents

Project Requirements

In this section, we will present the necessary libraries and tools that need to be present on the system in order to run the project. The specific python import libraries are all contained within a `requirements.txt` file that compiles them all and provides an easy way to set them up with one command using pip.

Backend Libraries and Requirements

Python (3.11)
The python version used in this project is 3.11.x. Older python versions may not guarantee compatibility. Newer, more recent versions of python should work in theory.

NVIDIA CUDA Toolkit (12.8)
The backend requires the NVIDIA CUDA Toolkit (with compatible NVIDIA GPU drivers) to enable GPU-accelerated model inference.

Transformers (4.49.0)
Library through which model is imported (via HuggingFace).

PyMuPDF4LLM (0.0.27)
Extracts and structures PDF content in an LLM-friendly format while preserving layout and reading order.

Langchain (0.3.27)
Framework that is pretty much used extensively throughout the project that provides abstractions for prompts, chains, agents, memory, and tool usage.

ChromaDB (1.0.20)
Vector database library for storing and retrieving embeddings using fast similarity search.

Rich (13.5.2)
Library used to print out clean debug outputs when prompting the model through the command line interface.

Unstructured (0.7.7)

Library that converts raw documents like PDFs and HTML into structured text elements for later processing.

FastAPI
Framework through which we implemented the HTML endpoints so that the frontend can communicate with the backend of the application.

Models Used

In this project, the backend imports 2 different models. One is the small language model that generates the output, but the second is the embedding model that provides the ability to convert the text chunks into their numerical vector representation.

Embedding Model

The embedding model we are using is `sentence-transformers/all-mpnet-base-v2` as it does well with research paper content compared to other embedding models that are more generalized. The model used can be changed if necessary through the config file in the backend directory (more on this later, see Project Structure section).

Small Language Model

Just like the embedding model, the language model used is specified in the config file and can also be changed. Earlier versions of the project used "`microsoft/phi-3.5-mini-4k-instruct`" but the current version is using "`microsoft/phi-3-medium-4k-instruct`" in order to mitigate the limited context size.

To use another model from Microsoft, the list of available importable models can be found at https://huggingface.co/microsoft/models. Other models from different developers can also be found on HuggingFace and imported into the project.

Note: Some of these models can get decently big in file size. Microsoft Phi-3 medium used in the project currently takes about 50GB of disk space, whereas the mini version still takes a hefty 7GB of disk space. On windows machines, the HuggingFace imports (downloads) are stored by default under `C:/Users/{user}/.cache/huggingface/hub`. If a model no longer needs to be used it can safely be deleted, but will have to be re-imported if that's not the case.

Frontend Frameworks and Libraries

Node.js (25.2.1)

In order to run the frontend Node.js must be installed. It is used to manage dependencies and to start the development server.

The frontend already has all dependencies stored inside a package.json file. Npm (included within the Node.js installation) is the dependency manager used to install those at project setup.

The next dependencies listed below are all specified within that file.

Frontend Frameworks:

- **React 18 (react, react-dom):** Core frontend UI framework.
- **React Router (react-router-dom):** Client-side routing.

Build and development tools:

- **Vite:** Development server and production bundler.
- **TypeScript:** Static typing and compilation step.
- **@vitejs/plugin-react:** React integration for Vite.

HTTP calls:

- **Axios:** Used for making HTTP requests to the backend.

Project Setup

The next section will deal with how to set up and run the project's backend and frontend components once the repository has been cloned.

Set-up commands (Backend)

Requirement installation

As mentioned earlier, for the backend python requirements a text file is provided under the backend subdirectory.

From the project's root: `cd backend`

To install the listed requirements within it simply run:
`python -m pip install -r requirements.txt`

Another command for torch components is also listed and required:
`pip install torch==2.5.1+cu118 torchvision torchaudio --extra-index-url https://download.pytorch.org/whl/cu118`

The instructions are provided under the README.md.

Building the Vector Database

Before prompting the model, we have to build the vector database. To do that simply put the desired PDFs under backend/data/test-data.

Notes:

- The repository does not include these directories or the test-data that we have been provided with, so they have to be created.
- The directory mentioned above that needs to be created is how it is set up in config.py by default and can be changed as desired).

The ingestion pipeline can then be called using: `python cli.py --ingest`

Every time the collection of PDFs expands, the ingestion pipeline can detect the added documents and add them to the database. However if PDFs have been deleted, the ingestion pipeline cannot remove the associated embeddings from the database. In this scenario the entire database (the backend/db) needs to be removed and rebuilt from scratch.

Note that this process can take a while, especially with higher PDF volumes. To improve the speed of the process, a future improvement to the project would be to add concurrency via multithreading to speed up the ingestion and storage of PDFs as chunks.

Running the Backend

There are two ways of running the backend. For debugging/testing purposes, the command line interface can be used by running cli.py.

Querying command: `python cli.py --query "{query}"`

The second method is by starting the api module using `python api.py`. The frontend will then need to be active in order to pass prompts.

Note: Running the CLI unloads the model from memory after the output is generated. Running the backend through the API module keeps it loaded for as long as the backend server is alive.

Set-up commands (Frontend)

For the frontend, the dependencies included within the package.json need to be installed before running the user interface.

From the project's root: `cd frontend/react-app`
To install the dependencies: `npm install`
To run the frontend simply use: `npm run dev`

The frontend will be available at **http://localhost:5173/** and can be opened in a browser.

Project Structure

This next section will present the codebase, each file, its methods, purpose and functionality, both for the backend and frontend.

Backend Code

api.py

Description

Handles loading the model and starting a locally hosted server which will handle POST requests sent from the frontend. Uses FastAPI and Uvicorn.

Method(s) Implemented

```python
def clean_answer_text(text: str) -> str:
    match = re.search(r'final answer:\s*(.*)', text, re.IGNORECASE | re.DOTALL)
    if match:
        after = match.group(1)
        paragraph = after.split('\n', 1)[0].strip()
        return paragraph
    return text.strip()

# Using post in order to send the query and get the answer
@app.post("/query", response_model=QueryResponse)
async def query_endpoint(req: QueryRequest):
    try:
        result = qa_chain(req.question)
        answer = result.get("output_text") or result.get("answer") or result.get("result")
        sources = [str(doc.metadata.get("source", "unknown")) for doc in result.get("source_documents", [])]

        clean_answer = clean_answer_text(answer['output_text'])

        return QueryResponse(answer=clean_answer, sources=sources)

    except Exception as e:
        logger.exception("Error while processing query: %s", e)
        raise HTTPException(status_code=500, detail=str(e))
```

This class implements a single POST endpoint (/query) that takes the input string, calls the backend chain and sends the answer back to the frontend through a Pydantic response model that serializes the answer and sources into a JSON payload.

Notes

In order to avoid loading the model twice and running into out of memory errors, an “on startup” method is defined to make sure the model gets loaded once before running the server. The specific way it is implemented currently is still supported in the current version of something, but will be fully deprecated with upcoming versions.

As the model generates its entire map-reduce chain, a method designed to trim the output text to only include the final answer has also been added. This should be removed if a fix to the output format of the model is found.

cli.py

Description
Handles debugging the querying chain once per run for debugging purposes through a command line interface. It handles loading the model, querying it once, printing the output cleanly and unloading the model. It also importantly handles building the vector database, which needs to be run when updates are made to the PDF library.

Method(s) Implemented
This class implements a helper method that handles pretty printing the output so that it is easier to read in the CLI.

Otherwise, the main entrypoint of this python file handles everything from parsing the arguments given to it (--ingest or --query depending on the use case) and handling the ingestion process by calling the relevant method, or handling the querying by setting up the QA chain and printing the output using the helper method.

Notes
The ingest command is run through: `python cli.py --ingest`
The querying command is run through: `python cli.py --query "{query}"`

config.py

Description
Stores the necessary constants, data paths and other important settings that are reused throughout the codebase.

List of Stored Items
- **Base Directory**
- **Data Directory** (where PDFs are stored)
- **ChromaDB Directory** (where the ingestion process will store the vector database)
- **ChromaDB Collection Name** (name of the database collection)
- **Model Name** (name of the imported model from HuggingFace)
- **Embedding Model** (name of the imported embedding model)
- **Chunk Size** (text chunk size in tokens), note that this value works well but is not necessarily optimal.

- **Chunk Overlap** (by how many tokens do consecutive chunks overlap), note that this value works well but is not necessarily optimal.

- **Temperature** (constant that controls randomness, lower values give safer and predictable answers, higher values give more varied and creative answers)
- **Top P** (Limits the model's choices to the most likely tokens whose probabilities add up to p. Smaller values lead to more focused answers while higher ones lead to diverse answers).
- **Repetition Penalty** (Higher values further discourage the repetition of words/phrases)
- **Max New Tokens** (maximum amount of new tokens model is allowed to generate)
- **Num Retrieve** (number of documents to retrieve)
- **Do Sample** (True/False constant that enables/disables randomness-based sampling. Disabling it leads to deterministic answers). When testing, the Do Sample parameter should be false for deterministic testing.

Notes
These constants can be modified as needed.
As mentioned earlier in the document, the names of the models need to match existing HuggingFace repository names that contain the models to be imported.

ingest.py

Description
Serves the purpose of handling the ingestion and local storage of the PDF research papers.

Method(s) Implemented

clean_text:

```python
def clean_text(text: str) -> str:
    """
    Clean the text extracted from PDFs:
    - Remove HTML tags
    - Normalize whitespace
    - Remove common page numbers
    """
    text = unicodedata.normalize("NFKC", text)
    text = re.sub(r"Page\s*\d+(\s*of\s*\d+)?", " ",  text, flags=re.IGNORECASE)
    text = re.sub(r"[\[\]{}*~^_]+", " ", text)
    text = re.sub(r"\s+", " ", text)
    text = re.sub(r"(?m)^\s*(Chapter|Section)\b.*$", "", text)
    return text.strip()

def list_pdfs(path: Path) -> List[Path]:
    return sorted([p for p in path.glob("**/*.pdf")])
```

This method performs the pre-processing of the PDF input data. This is important in order to normalize the input which means to remove chapter section titles, page numbers and other

elements of the PDFs that are not relevant for our model to have in its context. This vastly improves model performance.

list_pdfs:
Helper function used to list all PDFs in a directory, this is used to list the directories in our test directory for other methods to refer to the PDFs easily.

load_documents:

```python
def load_documents(pdf_paths: List[Path]):
    """
    Load PDFs and clean text.
    Add PDF filename as metadata for citation in QA.
    """
    docs = []
    for p in pdf_paths:
        loader = UnstructuredPDFLoader(str(p))
        raw_docs = loader.load()
        for doc in raw_docs:
            doc.page_content = clean_text(doc.page_content)
            # Add source PDF filename for citation
            doc.metadata["source"] = p.name
            docs.append(doc)
    return docs
```

This method looks at each PDF and uses UnstructuredPDFLoader to load all PDFs. The clean_text function is used to pre-process and then append the metadata of each PDF to a docs array. The point of appending metadata is to ensure that after the model outputs an answer, we can see which doc corresponds to which source. This is vital because we want to show the sources the model refers to when answering user queries.

split_documents:

```python
def split_documents(docs):
    """
    Split documents into semantic chunks with overlap.
    """
    splitter = RecursiveCharacterTextSplitter(
    chunk_size=config.CHUNK_SIZE,
    chunk_overlap=config.CHUNK_OVERLAP,
    separators=["\n\n", "\n", ".", " ", ""]
)
```

```python
    all_chunks = []
    for doc in docs:
        # Split and preserve metadata
        chunks = splitter.split_documents([doc])
        for i, chunk in enumerate(chunks):
            chunk.metadata["chunk_index"] = i
            all_chunks.append(chunk)
    return all_chunks
```

This method uses RecursiveCharacterTextSplitter (splits sentences by newlines/paragraphs) to go through all docs and to chunk/split them. We add metadata to each chunk to ensure we can source the relevant authors for any retrieved chunk for the model.

create_or_get_chroma_collection:

```python
def create_or_get_chroma_collection(client: chromadb.Client, name: str):
    # If already exists, get it.
    return client.get_or_create_collection(name=name)
```

Helper method to get the collection (list of documents we created / update) in our local ChromaDB instance.

get_existing_sources:

```python
def get_existing_sources(collection):
    """
    Return a set of all 'source' filenames already present in the collection.
    """
    existing = set()
    if collection.count() == 0:
        return existing

    results = collection.get(include=["metadatas"])
    for meta in results["metadatas"]:
        if meta and "source" in meta:
            existing.add(meta["source"])
    return existing
```

This method returns all existing source filenames in the current collection. It's looping through all collection elements metadata and returning their relevant source metadata.

add_pdfs_to_collection:

```python
def add_pdfs_to_collection(client, pdf_paths):
    """
    Load, clean, split, and embed a list of PDFs into the Chroma collection.
    """
    logger.info("Loading %d PDFs...", len(pdf_paths))
    docs = load_documents(pdf_paths)

    logger.info("Splitting into chunks...")
    chunks = split_documents(docs)

    logger.info("Embedding and adding documents to Chroma...")
    embeddings = SentenceTransformerEmbeddings(model_name=config.EMBEDDING_MODEL)
    vector_store = Chroma(
        client=client,
        collection_name=config.CHROMA_COLLECTION_NAME,
        embedding_function=embeddings
    )

    ids = [str(uuid.uuid4()) for _ in chunks]
    vector_store.add_documents(documents=chunks, ids=ids)

    logger.info("Ingest complete. Added %d chunks from %d PDFs.",
                len(chunks), len(pdf_paths))
```

This helper method runs the loading of documents, the splitting of documents into chunks and then instantiates a local instance of ChromaDB to use as a vector store. We create an instance of IDs to store our chunks and add them to the DB. Unique IDs are created due to ChromaDB requiring each individually stored document to have their own unique ID.

ingest_all:

```
def ingest_all():
    pdfs = list_pdfs(Path(config.DATA_DIR))
    if not pdfs:
        logger.warning("No PDFs found in %s", config.DATA_DIR)
        return

    client = chromadb.PersistentClient(path=config.CHROMA_PERSIST_DIR)
    collection = create_or_get_chroma_collection(client, config.CHROMA_COLLECTION_NAME)
    logger.info("Collection '%s' already has %d items", config.CHROMA_COLLECTION_NAME, collection.count())

    existing_sources = get_existing_sources(collection)
    new_pdfs = [p for p in pdfs if p.name not in existing_sources]

    if not new_pdfs:
        logger.info("No new PDFs to ingest.")
        return

    logger.info("Found %d new PDFs: %s", len(new_pdfs), [p.name for p in new_pdfs])
    add_pdfs_to_collection(client, new_pdfs)
```

This method runs the add_pdfs_to_collection method if there is a new PDF found, otherwise it does nothing. It allows for adding of PDFs dynamically to the current list of ingested PDFs.

model.py

Description

This file is used to import the model and configure it.

Methods

This python file implements a load method that uses AutoModelForCausalLM from the transformers library to import the pre-trained Phi-3 model. It uses a tokenizer and pipeline object to help instantiate the model with its configured parameters in config.py.

The import itself makes sure that optimizations are applied to the model when importing, notably low_cpu_mem_usage and load_in_4bit are both set to true, the latter being the quantization parameter.

qa_service.py

Description

This file is used to interface with the model which includes creating the map and reduce prompts and building the question and answer chain to communicate with the model.

Methods

```
# --- Map Prompt ---
map_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "You are a technical research assistant. "
     "You must use **only** the information in the provided context to answer the question. "
     "You **cannot** use outside knowledge, assumptions, or hallucinate facts. "
     "If the context does not provide any relevant information, respond exactly: NO RELEVANT INFO."),
    ("user",
     "Context:\n{doc_text}\n\nQuestion:\n{question}\n\n"
     "Instructions:\n"
     "1. If the context directly answers the question, summarize the answer clearly (max 150 tokens).\n"
     "2. If the context does not answer the question but provides background or related information, "
     "summarize how the context relates to the question (max 150 tokens).\n"
     "3. If the context is unrelated, respond exactly: NO RELEVANT INFO.\n\nAssistant:\nSummary:")
])

# --- Reduce Prompt ---
reduce_prompt = ChatPromptTemplate.from_messages([
     ("system",
     "You are a precise technical summarizer. Use only the provided summaries."),
    ("user",
     "Partial answers (each with its source):\n{partial_answers}\n\n"
     "Question:\n{question}\n\n"
     "Instructions:\n"
     "1. Discard any entries that are 'NO RELEVANT INFO.'\n"
     "2. Choose the most relevant and complete answer that directly addresses the question.\n"
     "3. If multiple are similar, merge minimally while preserving factual clarity.\n"
     "4. If none directly answer, respond exactly: NO RELEVANT INFO.\n\n"
     "Final Answer:")
])
```

The map prompt is used to initially take each chunk and make the model summarize them individually. The reduce prompt is used to take all those summaries created by the model and create a contextually relevant and intelligent answer based on those summaries.

The prompts were made through trial and error. A few important notes regarding the prompts:

- The model must be explicitly told to not reference external content, otherwise it will search on the web for an answer

- You must assign a role to the model, this is a common documented means of improving model performance
- Strict instructions for output format, even in the case of no relevant response must be given for the model to follow

get_vector_store:

```python
# --- Vector store ---
def get_vector_store():
    client = chromadb.PersistentClient(path=config.CHROMA_PERSIST_DIR)
    embeddings = SentenceTransformerEmbeddings(model_name=config.EMBEDDING_MODEL)
    return Chroma(
        client=client,
        collection_name=config.CHROMA_COLLECTION_NAME,
        embedding_function=embeddings
    )
```

This method is a helper function used to refer to our local instance of ChromaDB.

MapReduceQAWrapper Class:

```python
# --- Wrapper class ---
class MapReduceQAWrapper:
    def __init__(self, retriever, map_reduce_chain):
        self.retriever = retriever
        self.chain = map_reduce_chain

    def __call__(self, query: str):
        docs = self.retriever.get_relevant_documents(query)
        logger.info("Retrieved %d documents for query.", len(docs))

        answer = self.chain.invoke({
            "input_documents": docs,
            "question": query
        })

        return {
            "result": answer,
            "answer": answer,
            "source_documents": docs
        }
```

This class was created because we needed a way of creating a QA chain to interface with our model. It's a wrapper class used to assist with the implementation of our Map-Reduce chain.

build_retrieval_qa:

```python
# --- Build the retrieval QA ---
def build_retrieval_qa(llm, k: int = None):
    vector_store = get_vector_store()
    retriever = vector_store.as_retriever(
         search_type="similarity",
        search_kwargs={
            "k": k
        }
    )

    # Map-Reduce chains
    map_chain = LLMChain(llm=llm, prompt=map_prompt)
    reduce_llm_chain = LLMChain(llm=llm, prompt=reduce_prompt)

    combine_documents_chain = StuffDocumentsChain(
```

```python
        llm_chain=reduce_llm_chain,
        document_variable_name="partial_answers"
    )

    reduce_documents_chain = ReduceDocumentsChain(
        combine_documents_chain=combine_documents_chain,
        token_max=2000
    )

    map_reduce_chain = MapReduceDocumentsChain(
        llm_chain=map_chain,
        reduce_documents_chain=reduce_documents_chain,
        document_variable_name="doc_text"
    )

    logger.info("Initialized Map-Reduce RetrievalQA chain successfully.")
    return MapReduceQAWrapper(retriever, map_reduce_chain)
```

We build a retriever object which is how we perform similarity search to dynamically search our vector DB for k chunks that most closely match the given user query.

We then use StuffDocumentsChain, ReduceDocumentsChain and MapReduceDocumentsChain to create our QA chain to interact with the model.

Other Files (Backend)

LoRA-archived

Archive directory containing the implementation and results from the LoRA fine-tuning experiment. Does not affect or interact with the project in any way, has only been archived to keep it recorded.

requirements.txt

Simple text file that contains all of the required libraries to be installed with pip when setting up the project.

Frontend Components

App.tsx / App.css

The main component of the react application, that imports all other sub-components and handles the home page and chat page routing, as well as specifying the HTML format of the home page. The App.css handles the styling for the frontend as a whole.

Chat.tsx / InputBox.tsx

Components that are displayed on the chat page. They implement the chat and input box components as well as their frontend logic for sending and displaying user chats and model responses.

main.tsx

This main component simply calls the app component.

Improvements Needed and Limitations

This section of the document will detail some important items from the project that need addressing, confirmation or improvement, as well as limitations with respect to the current implementation.

Map Reduce

Improvements to the current implementation of the map-reduce chain are needed. This does not exclude finding a more efficient implementation using a different chain, or a different architecture altogether to process the answer.

To start, the map-reduce chain consumes a significant amount of tokens that exceeds what Phi-3 Medium can handle. This is due to map reduce's partial summaries accumulating a significant amount of text. This effectively means that the tokens passed through to the final answer exceed the model's context window. This leads to some answers not generating at all, and presumably impacts the accuracy of ones that end up generating.

There is also the heavy consumption of output tokens, where the model re-generates all of the intermediate summaries from the map reduce chain including the re-printing of the prompt templates.

Runtime VRAM Usage

Even with the smallest model available, the runtime of the model consumes a significant amount of GPU memory. But while that case can be manageable, the current use of the medium variant of Phi-3 significantly increased VRAM usage. To give actual numbers, a desktop 4070 Ti with 12

GB of dedicated memory manages to output the answer in around a minute, whereas a weaker, but still significantly performant laptop 3060 with 6GB of memory takes double or triple that time.

On the system with the 4070 Ti, GPU memory was completely saturated, necessitating offloading of the remaining memory usage to the system's RAM, significantly decreasing the performance of the runtime. This happens even with 4-bit quantization applied.

The use of the medium variant was made to try and mitigate context window issues as well as testing heavier variants of Phi-3.

As a related note, the API's runtime over lengthy periods of time needs to be evaluated to check for any degradation of performance over time.

Output Accuracy

As mentioned previously the output echoes back the prompts and the intermediate steps which is the first point that needs to be addressed.

As for the accuracy of the content itself, the model often fails to capture the specific details that the papers stress, or the context in which some concepts are used. The output often specifies a good definition for a query about a specific concept, however the model cannot apply that to the specific Computational Electromagnetics field, and tends to generalize its answer.

Moreover, the model cannot yet specifically answer questions about the structure of a research paper (specific sections, table of contents), or provide a good summary of a given paper, because it only considers similarity between the query and the chunks of text. It does not capture the context behind the question.

It is to be noted that the items listed above are caused in part by the fundamental limitations with respect to the small language model itself (reasoning limits, context window size, etc.). Expecting detailed and sharp outputs at the level of a LLM (such as ChatGPT) out of a small language model is obviously unrealistic. With this in mind, we think improvements can still be made to the RAG pipeline, but only up to a certain point.

Frontend Codebase

With respect to the frontend, a code refactoring would be needed should this project be expanded, specifically due to the home page not being its own tsx component. This would help maintainability greatly.

App.css also contains the entirety of the frontend's styling specifications, this should also be broken down per component.

Conclusions

What The Project Delivered

The project delivered a functional POC that demonstrates that we can interact with an open-source SLM to create answers to user queries based on an entirely new context consisting of research paper PDFs. We showed that this can be done locally with local ingestion and running of the model.

What The Project Failed To Do

The project is not easily scalable. Since all ingestion and running of the model is done locally, the user requires a GPU to properly run the model. The main issues with the project is that a SLM is too large to run on a local machine and is not efficient in most cases. While this was a core requirement of the project, it should be reevaluated. Instead, the model should be run using a client-server architecture and cloud services should be involved. In fact, the entire storage of the PDFs should be completed using a cloud instance of ChromaDB rather than a local instance of ChromaDB.

In order to ensure that users have the best possible experience with the application and that as many users can access it as possible, it is imperative to use cloud services for the running of the model and storage. While the project's current implementation of RAG is functionally correct, it can be greatly improved for production purposes. One potential improvement is to take user sentences and to perform a ranking system to rank certain words as more important than others in the context of similarity search. For example, nouns are more important in their sentences since they can refer to technical terms.

We also noticed that there are token limit errors that can occur if the context is too large which happens too quickly (only 3 documents are currently being passed to the model). Given the size of the PDF papers and technical content, using a cloud service can provide the incentive to

comfortably use a larger SLM than phi-3-mini-instruct or phi-3-medium which can provide much greater performance.

Instead of similarity search, a hybrid search between similarity search and MMR search would be worth exploring to obtain a mix of searching strictly based on similarity of keywords and diversity in the result set of matched documents.

Next Features To Implement

All storage of PDF documents should be done on the cloud and so should the running of the model. Afterwards, it would be vital to improve the current RAG pipeline to have more thorough pre-processing of PDF input data, to improve the current map-reduce method and to optimize model parameters. In fact, while we brute-force tested and obtained a good chunk size and chunk split overlap of the model, these parameters should be tested to be optimal via hyper-parameter searching.

In the frontend, a user should be able to upload a PDF and have it added to the backend. Moreover, the current model is not a conversational AI meaning that it does not retain context of the conversation. Instead, the current model is only answering user queries with contextually relevant answers. It would be critical to implement conversational AI so that the user can continuously interact with the model while it retains complete context of the conversation.